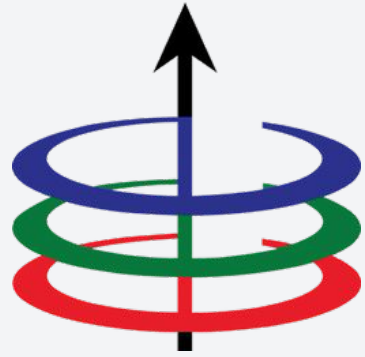




Lecture 8

Disjoint Set Data Structure (Union-Find)

EEE 121 2s2526

Steven S. Sison

SP1 Presentation

- Presentation for SP1 is moved to **April 29, 2026** during class hours.
 - This will be held F2F regardless of any strike.
- Instructions:
 - Sign-up using this link: <https://bit.ly/41Jqr2m>
 - Each group will have a total of 10 minutes to present (8mins presentation, 2 mins Q&A). Exceeding the allotted time will lead to deductions.
 - After presenting, submit your deliverables in the UVLe submission bin. Failure to do so will lead to a score of zero even if you have already presented.
- Good luck! :D

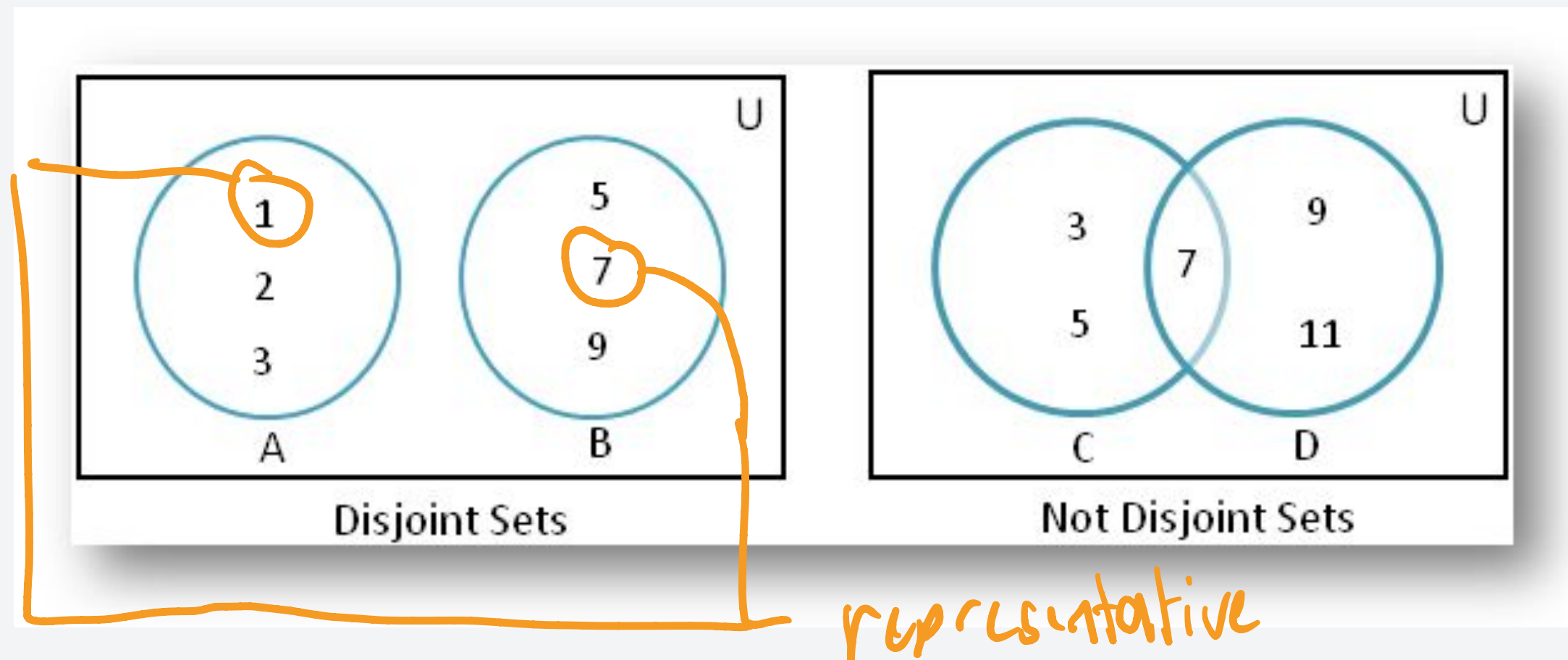
- Disjoint Set DS (Union-Find)
- Implementation
- Applications

- **Disjoint Set DS (Union-Find)**
- Implementation
- Applications

Disjoint Sets

A disjoint set data structure, also known as Union-Find, is a collection of sets $S = \{S_1, S_2, S_3, \dots, S_N\}$ that is disjoint and dynamic.

- A given subset S_A and S_B is disjoint if $S_A \cap S_B = \emptyset$.
- The dynamic property means the contents of a set S can change.
- Each set is identified by some member of the set called the **representative**.



Disjoint Sets - Operations

Operations:

→ initialization

→ representative

1. **MakeSet(N)**: creates N sets where each set consists of only one element.
2. **Union(x,y)**: unites the sets that contain the elements x and y into a new set. The sets of x and y can then be removed.
3. **FindSet(x)**: returns the representative of the unique set containing element x.

Make Set (3)

①

1

②

2

③

3

Union (1, 2)

①, 2

1

FindSet(2) = 1

③

3

Union (2, 3)

①

~~②, 3~~

not ng

①, 2, 3

correct

Disjoint Sets - Operations

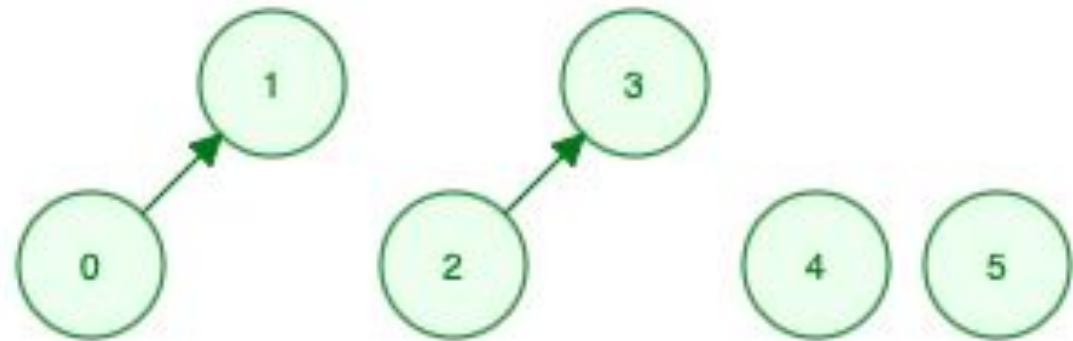


MakeSet(6) - creates 6 elements (0,1,2,...,5)

Disjoint Sets - Operations



MakeSet(6) - creates 6 elements (0,1,2,...,5)



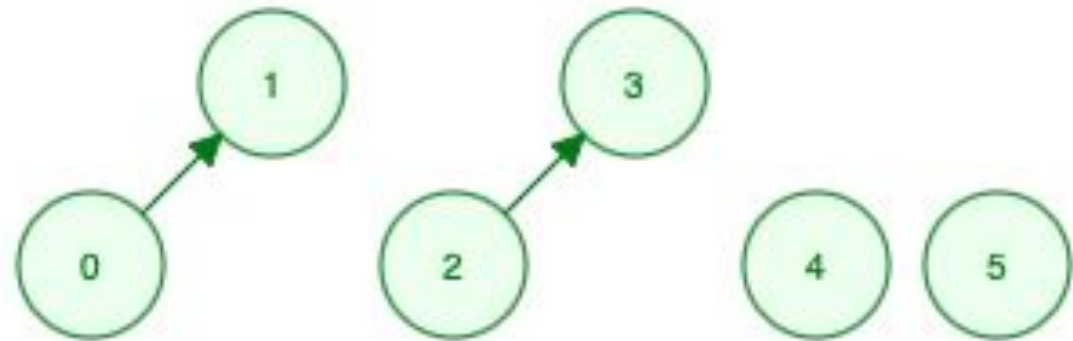
Union(0,1) - combines 0 and 1 with 1 as representative

Union(2,3) - combines 2 and 3 with 3 as representative

Disjoint Sets - Operations

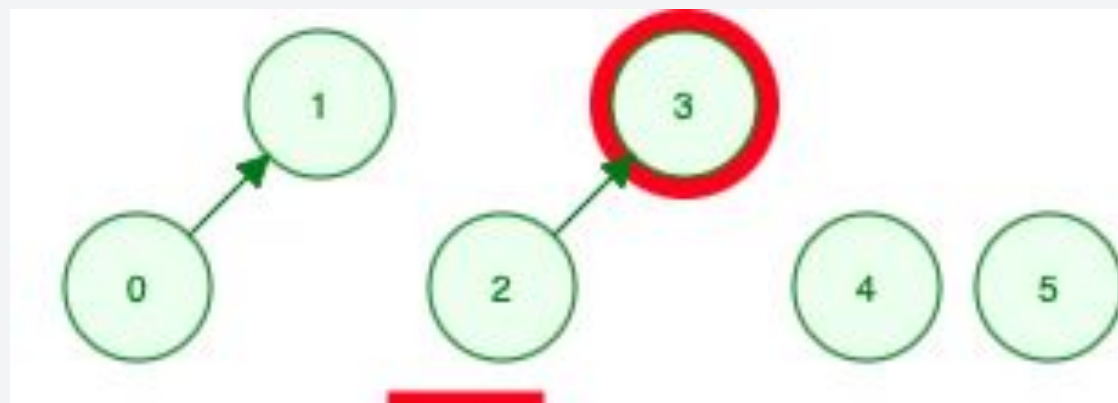
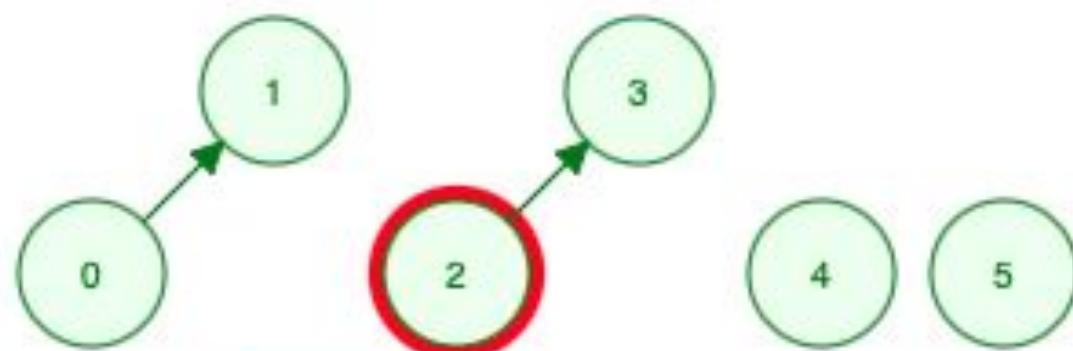


MakeSet(6) - creates 6 elements (0,1,2,...,5)



Union(0,1) - combines 0 and 1 with 1 as representative

Union(2,3) - combines 2 and 3 with 3 as representative



Find(2) - looks for 2, then returns the representative (3)

Why do we need Disjoint Sets?

Disjoint Set data structures are particularly useful in applications where you need to:

- Determine if two elements belong to the same set (Find)
- Combine two subsets into one single subset (Union)

Why do we need Disjoint Sets?

Disjoint Set data structures are particularly useful in applications where you need to:

- Determine if two elements belong to the same set (Find)
- Combine two subsets into one single subset (Union)

Where do we see this?

Why do we need Disjoint Sets?

Disjoint Set data structures are particularly useful in applications where you need to:

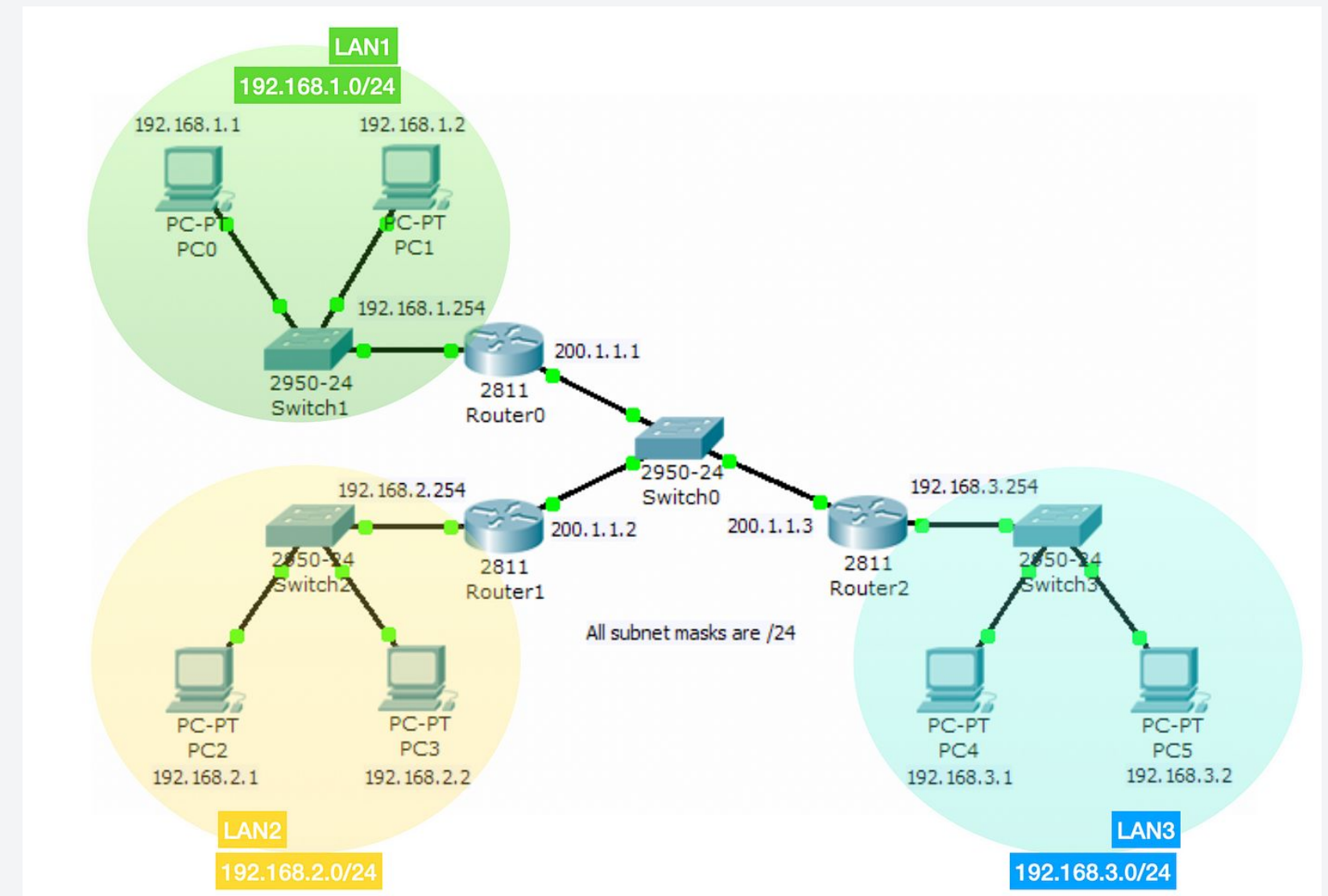
- Determine if two elements belong to the same set (Find)
- Combine two subsets into one single subset (Union)

Where do we see this?

- **Internet and Computer Networks:**

Using disjoint sets, we can determine if two devices are from the same network (subset).

- This information helps us to optimize routing protocols to these devices.



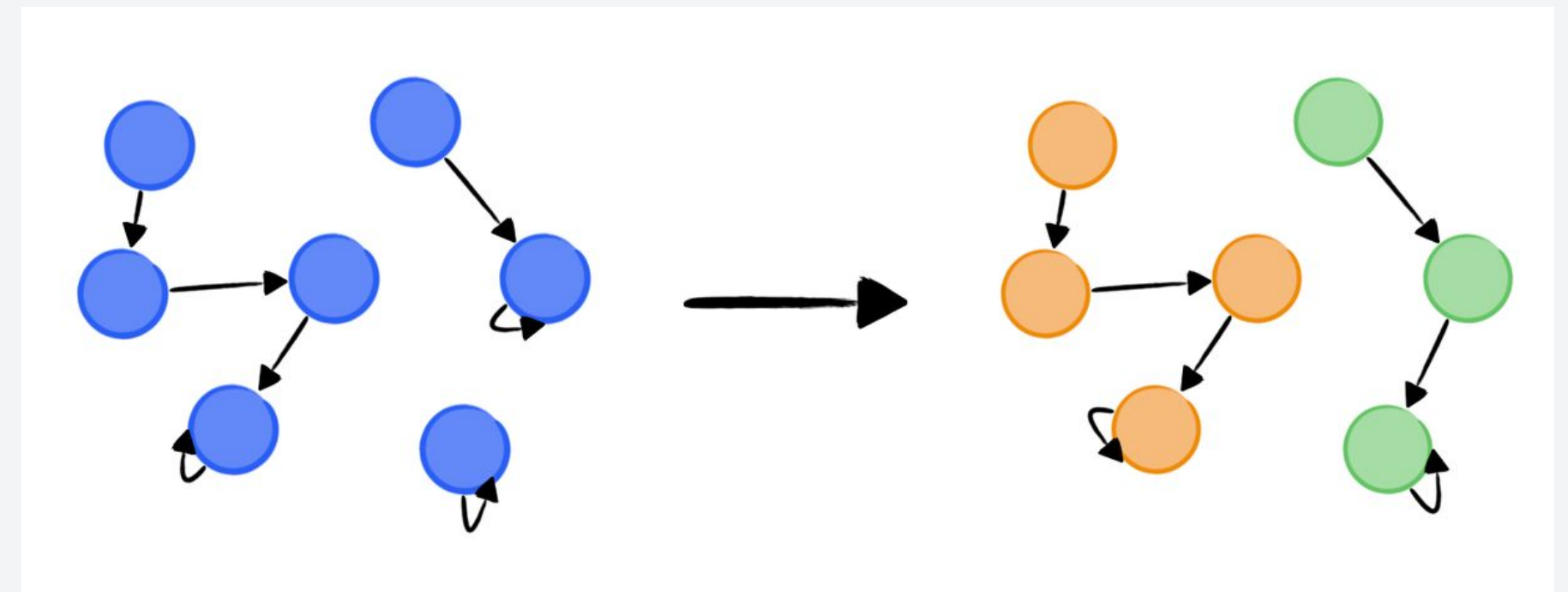
Why do we need Disjoint Sets?

Disjoint Set data structures are particularly useful in applications where you need to:

- Determine if two elements belong to the same set (Find)
- Combine two subsets into one single subset (Union)

Where do we see this?

- **Road Networks:** Using disjoint sets, we can use the Find operation to determine if two cities are connected by a road or not. Additionally, we can use the Union operation to combine two disjoint road networks.



- Disjoint Set DS (Union-Find)
- **Implementation**
- Applications

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

$\text{FindSet}(x) \Rightarrow \text{FindSet}(10) = 2 // \Rightarrow O(1)$ Array Access

idx	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
rep	1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25

rep = 1

2, 6, 10, 18, 22

rep = 2

4, 12, 20

rep = 4

8, 24

rep = 8

16

rep = 16

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- **Element 1, 2, 4, 8, and 16 are the representatives.**
- This means that element 3 belongs to set 1, element 10 belongs to set 2, so on and so forth.

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- To find the set of an element x , we use $\text{FindSet}(x)$.
 - $\text{FindSet}(3) =$
 - $\text{FindSet}(10) =$

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- To find the set of an element x , we use $\text{FindSet}(x)$.
 - $\text{FindSet}(3) = 1$
 - $\text{FindSet}(10) = 2$
- **What do you think is the time complexity of $\text{FindSet}(x)$?**

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- To find the set of an element x , we use $\text{FindSet}(x)$.
 - $\text{FindSet}(3) = 1$
 - $\text{FindSet}(10) = 2$
- $\text{FindSet}(x)$ is fast with $O(1)$ time complexity. Why?

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- To find the set of an element x , we use $\text{FindSet}(x)$.
 - $\text{FindSet}(3) = 1$
 - $\text{FindSet}(10) = 2$
- $\text{FindSet}(x)$ is fast with $O(1)$ time complexity. Why? **Accessing elements in an Array**

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

Convert elements of set 2 to set 1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1
	1				1				1				1				1				1			

In this example:

- But what about union? Union (10, 12)

$O(n)$

$\left\{ \begin{array}{l} \text{4x FindSet}(10) = 2 \quad O(1) \\ \text{6x FindSet}(12) = 4 \quad O(1) \end{array} \right.$

1x Traversal array to check elements belonging to 2 or 4 $O(n)$

How do we implement Disjoint Sets?

Intuitively, we can implement disjoint sets using two arrays – the first array contains the elements while the second array contains the representative that the element belongs to.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1

In this example:

- But what about union?
 - We need to traverse the whole array to find the elements of a set and change its representative.
- **It would take $O(N)$**

How do we implement Disjoint Sets?

So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.

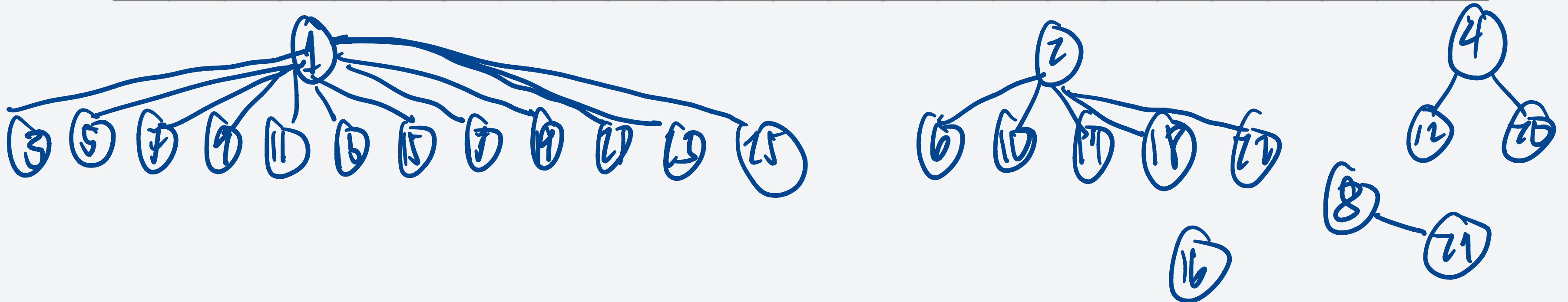
How do we implement Disjoint Sets?

So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.

Let's try it with the previous example

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1



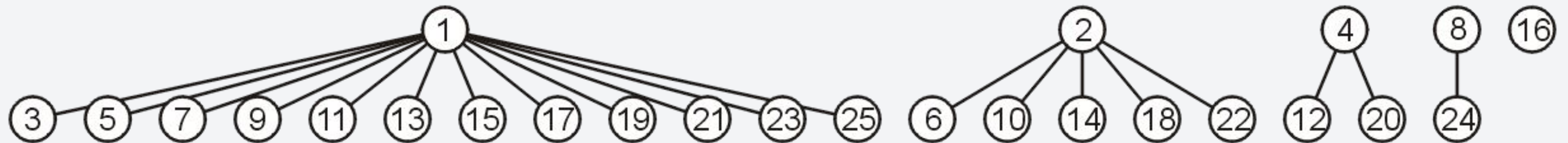
How do we implement Disjoint Sets?

So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.

Let's try it with the previous example

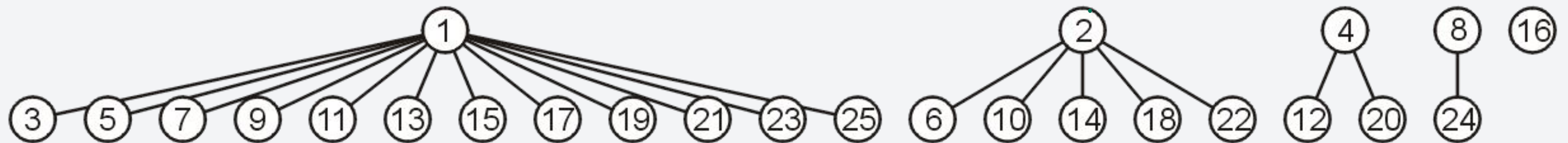
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16	1	2	1	4	1	2	1	8	1



How do we implement Disjoint Sets?

So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.

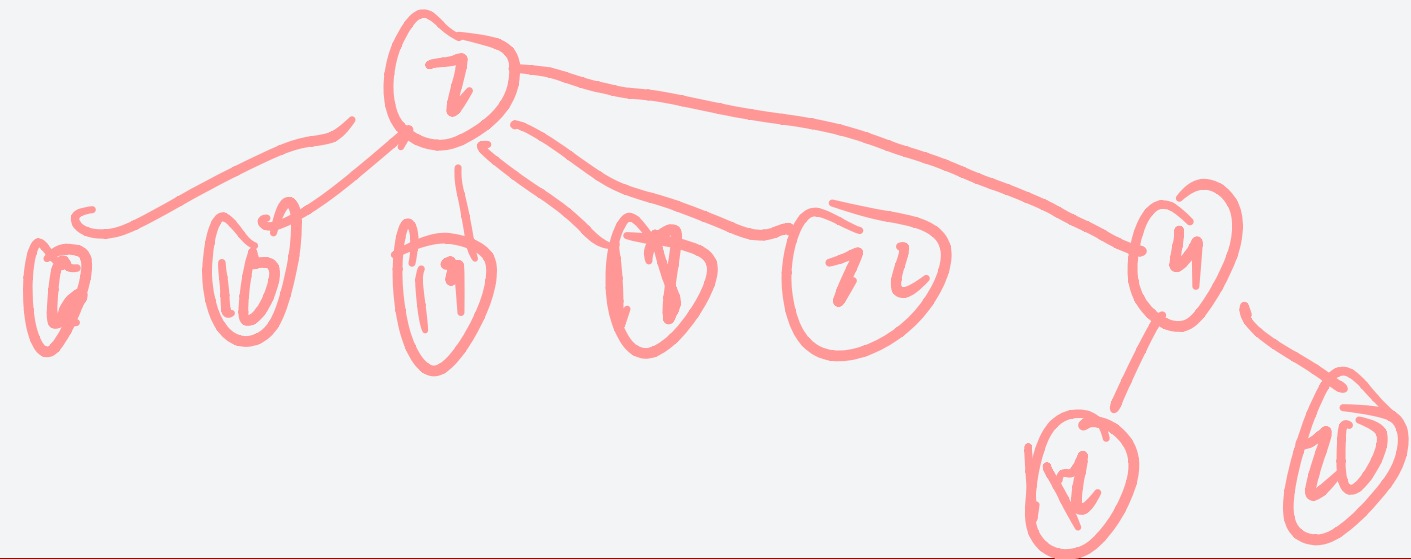
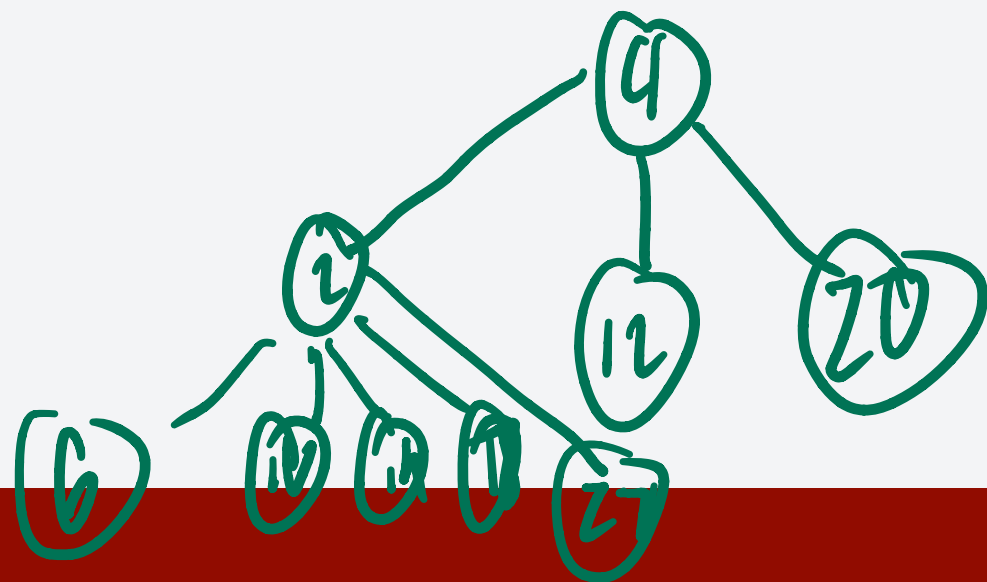


How is the union operation then performed? *Union (10, 12)*

- FindSet(10) = 2, FindSet(12) = 1

• $\text{parent}[2] = 4$

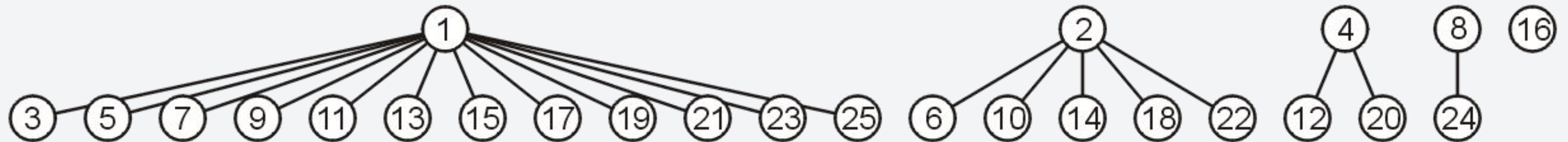
• $\text{parent}[4] = 2$



How do we implement Disjoint Sets?

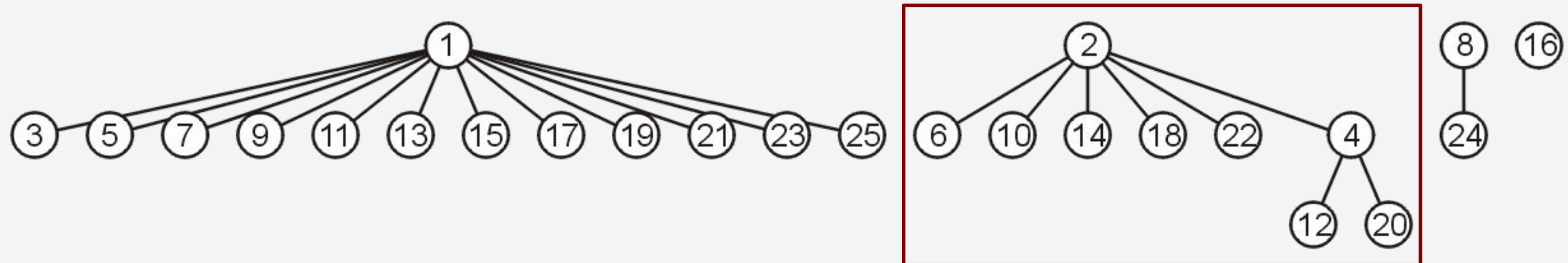
So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.



How is the union operation then performed?

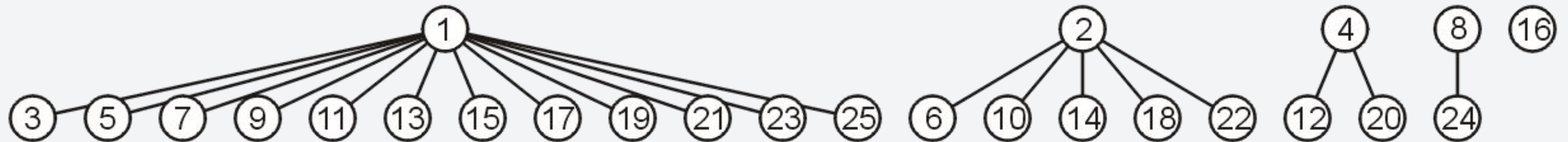
Attach the root of one tree to the root of another.



How do we implement Disjoint Sets?

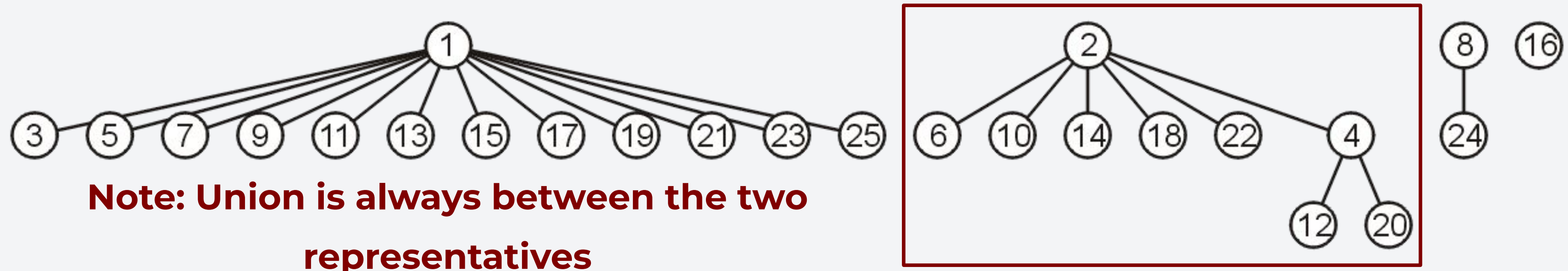
So how do we speed it up? We can represent each disjoint set as a tree.

- The root of the tree acts as the representative of the set.



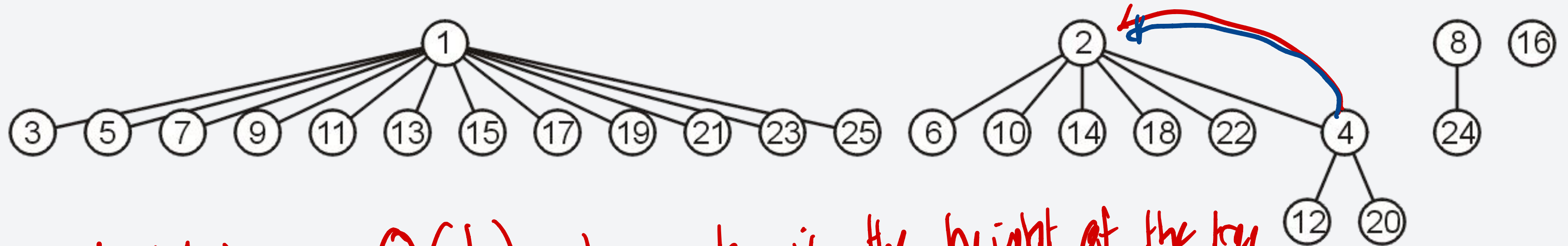
How is the union operation then performed?

Attach the root of one tree to the root of another.



How do we implement Disjoint Sets?

Given these structure, can you determine the time complexity of find and union?



FindSet(x) $\Rightarrow O(h)$ where h is the height of the tree

Union(x, y)

$\hookrightarrow$ FindSet(x)

$O(h_x)$

$\hookrightarrow$ FindSet(y)

$O(h_y)$

$\hookrightarrow$ parent[x] = y or parent[y] = x

$O(1)$

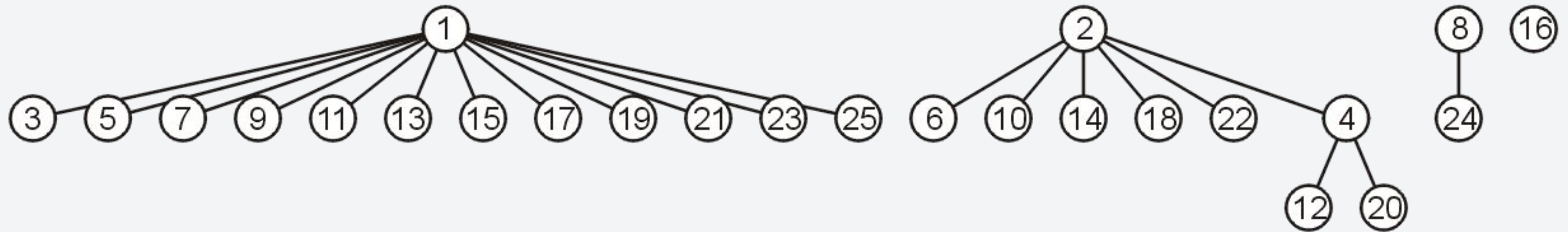
$O(h_x + h_y)$

$\Rightarrow O(h)$

bounded by the max h tree

How do we implement Disjoint Sets?

Given these structure, can you determine the time complexity of find and union?



Both FindSet(x) and Union(x,y) have $O(h)$ time complexity where h is the height of the tree.

- FindSet(x): From the element, we need to traverse upwards to find the root.
- Union(x,y): From the element x and y , we need to find their root nodes separately making the time complexity $O(h_x + h_y) = O(h)$.

Code Implementation for Disjoint Sets

Initialization:

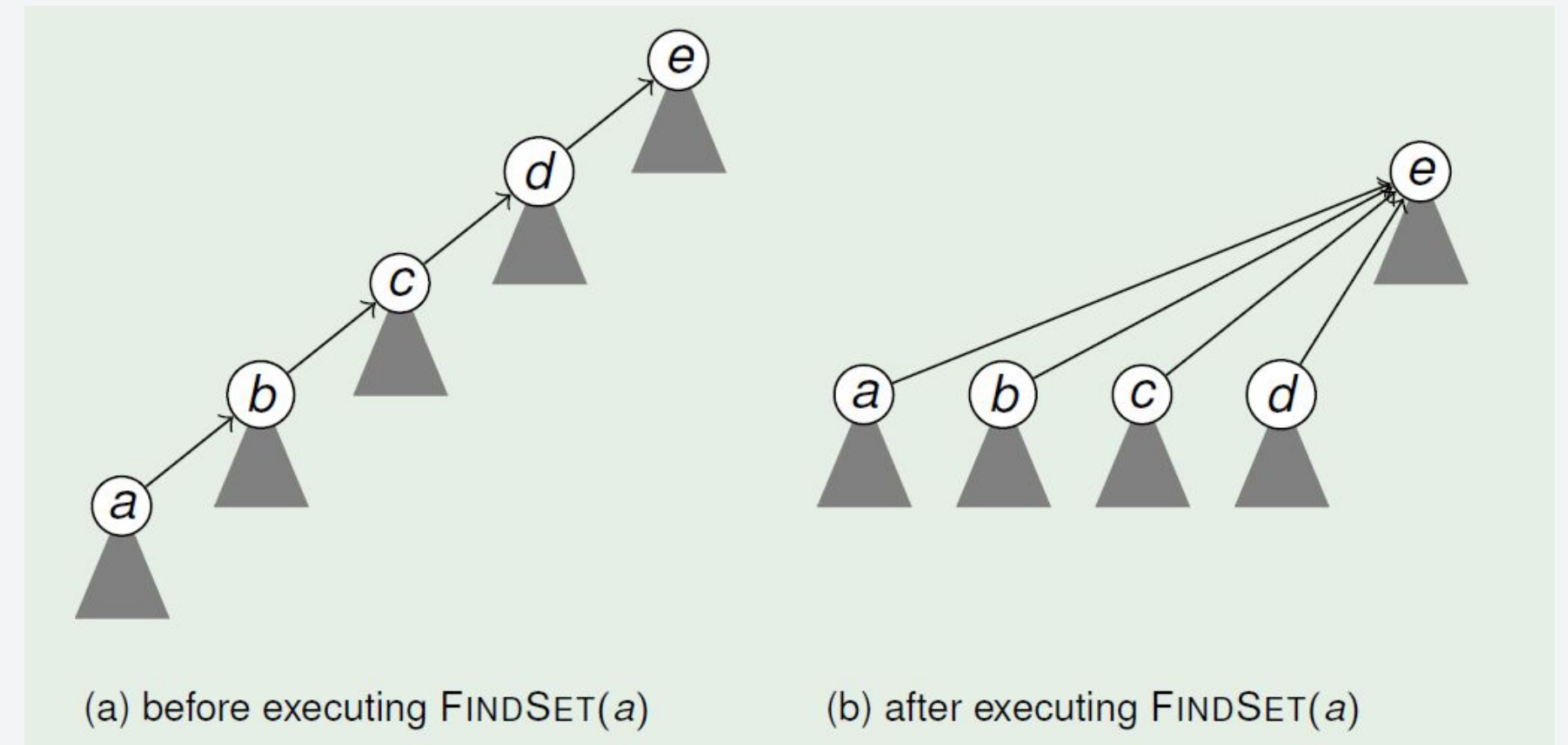
```
void MakeSet(int N) {  
    for(int i = 0; i < N; i++)  
        parent[i] = i; //representative of the set is itself  
}
```

- MakeSet(N) creates a parent array where each index is an element, and the value that the index contains is the representative of the element.
- To become a root node, $\text{parent}[i] == i$.

Code Implementation for Disjoint Sets

The FindSet(x) function:

```
int FindSet(int x) {  
    if(parent[x] != x) {  
        x = FindSet(parent[x]);  
        parent[x] = x;  
    }  
    return x;  
}
```

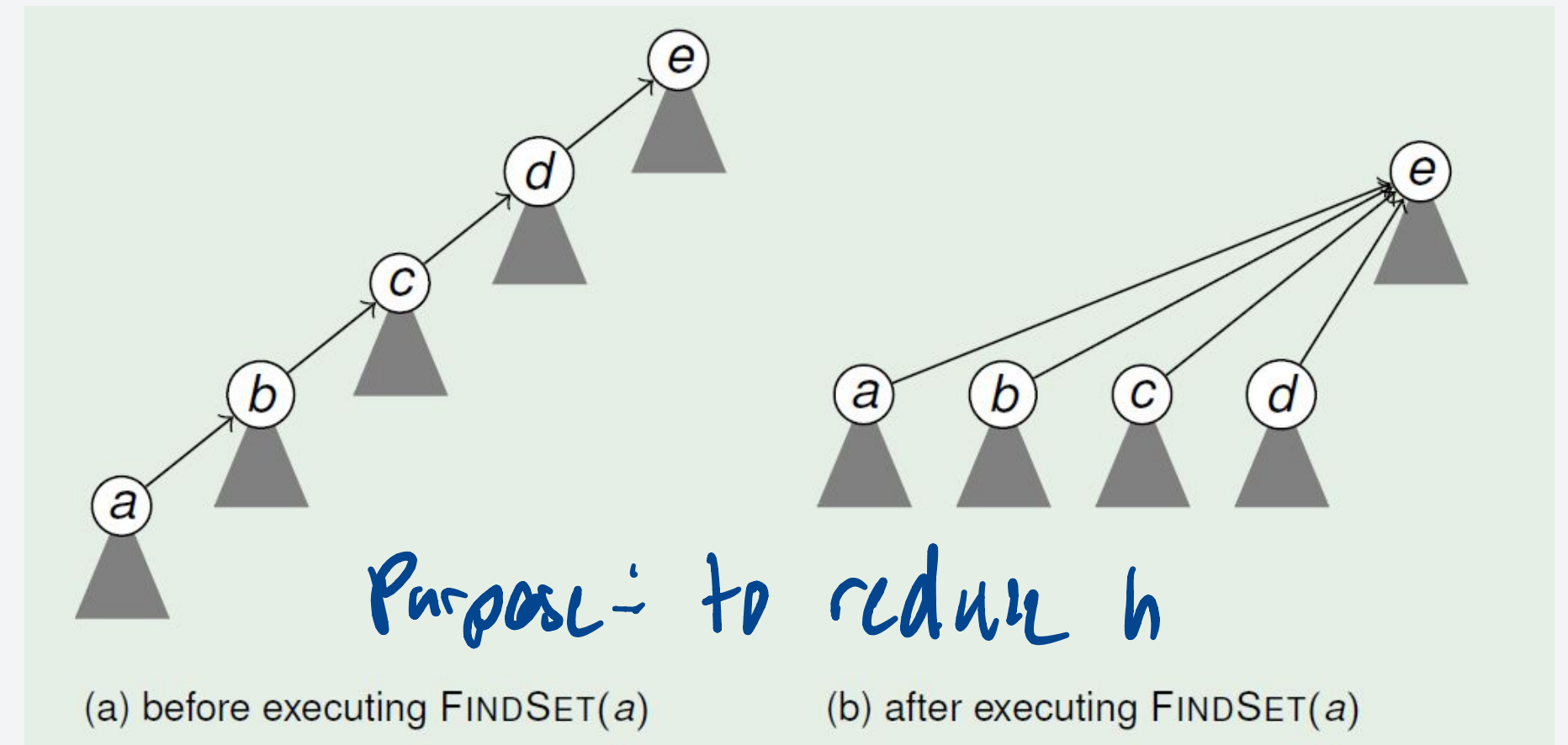


- The FindSet(x) operation is performed recursively to find the root node of a set.

Code Implementation for Disjoint Sets

The FindSet(x) function:

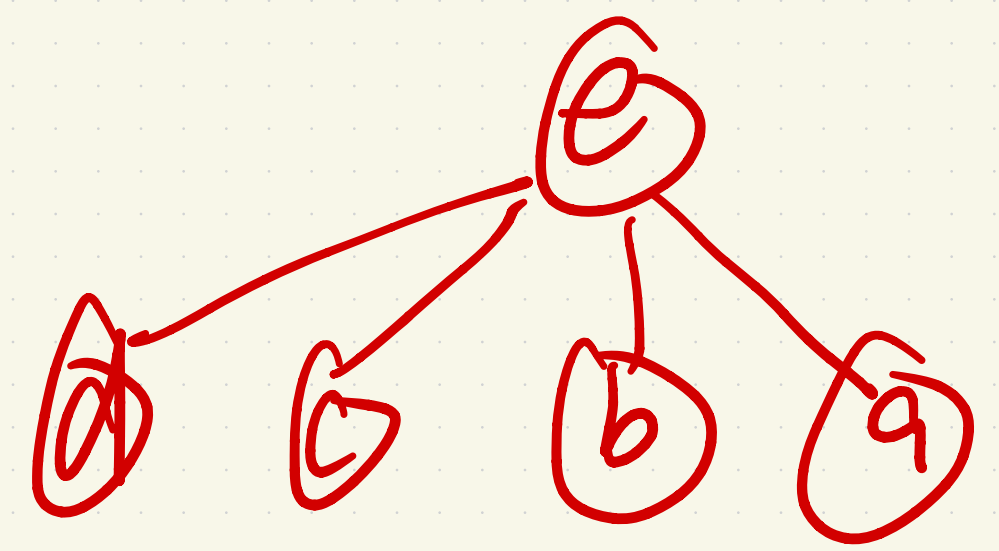
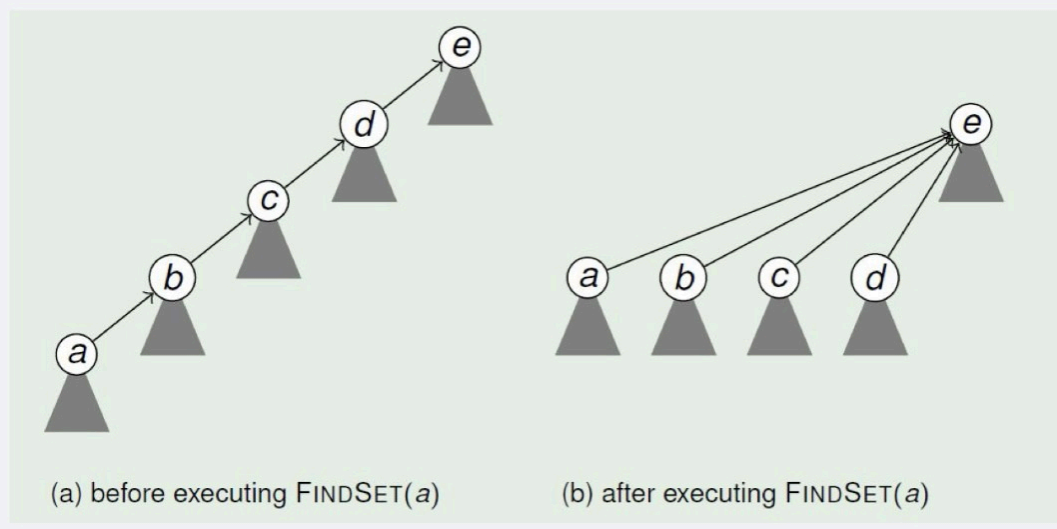
```
int FindSet(int x) {  
    if(parent[x] != x) {  
        x = FindSet(parent[x]);  
        parent[x] = x;  
    }  
    return x;  
}
```



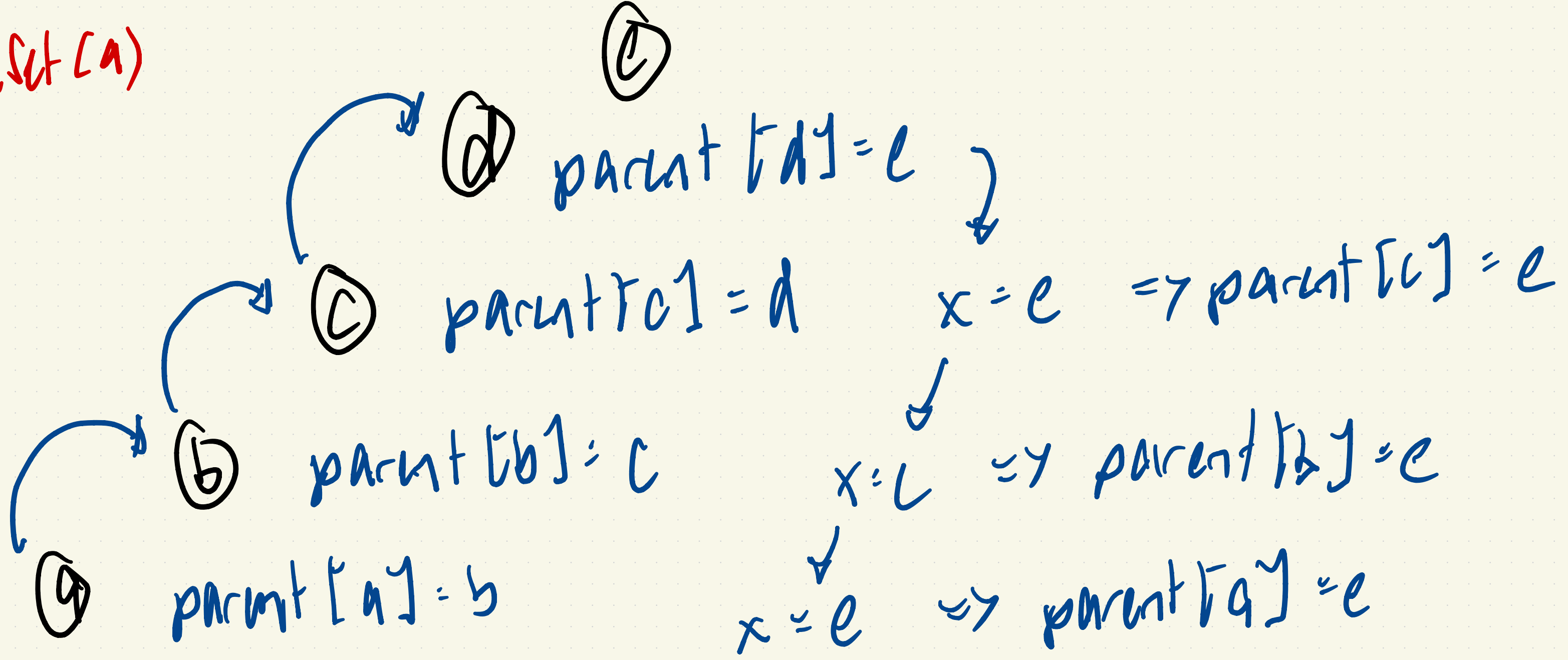
- The FindSet(x) operation is performed recursively to find the root node of a set.
- Additionally, it has **path compression** built in wherein each node on the path will be made to point directly to the root node.
 - Recall: The union of two sets creates attaches one tree to another. The path compression will compress this tree as seen in the image.

The FindSet(x) function:

```
int FindSet(int x) {  
    if(parent[x] != x) {  
        x = FindSet(parent[x]);  
        parent[x] = x;  
    }  
    return x;  
}
```

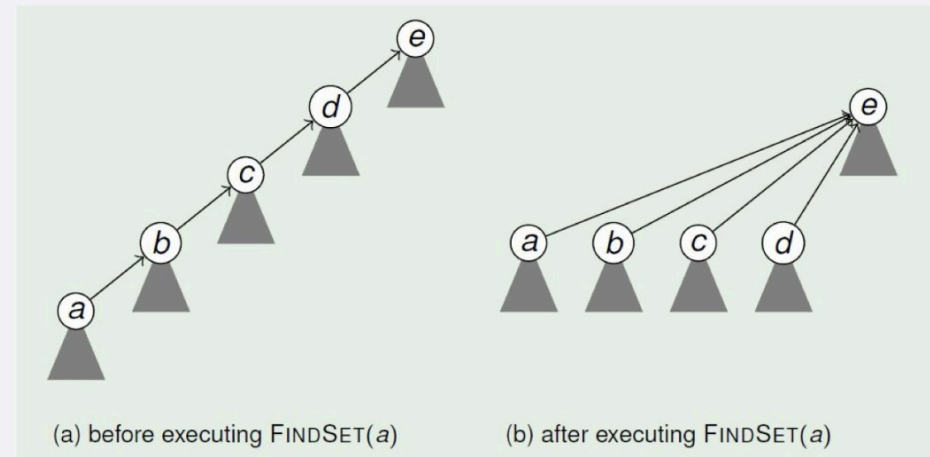


FindSet(a)

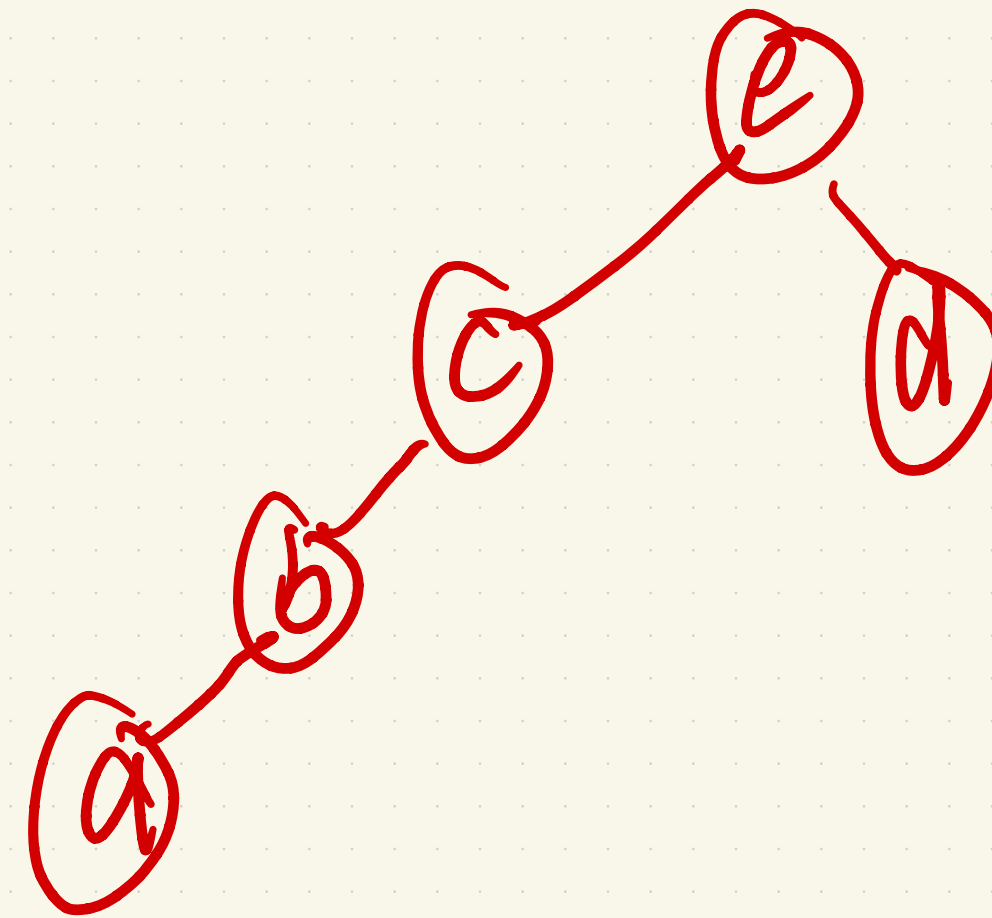


The FindSet(x) function:

```
int FindSet(int x) {  
    if(parent[x] != x) {  
        x = FindSet(parent[x]);  
        parent[x] = x;  
    }  
    return x;  
}
```



FindSet(c)



e
 $d \rightarrow \text{parent}[d] = e, x = e$
 $\Downarrow$
 $c \rightarrow \text{parent}[c] = d, x = d \Rightarrow \text{parent}[c] = e$

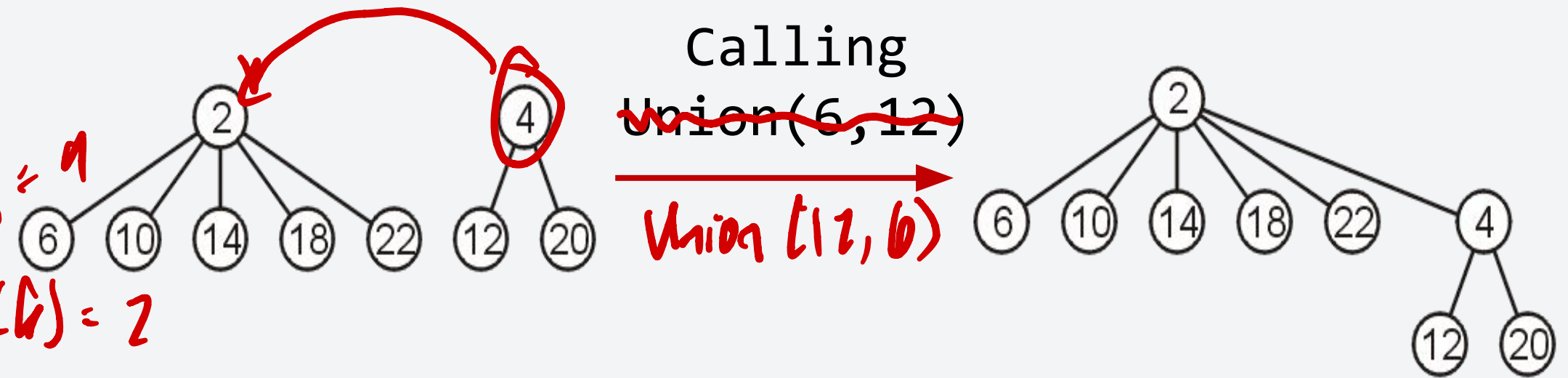
$\text{parent}[a] = b$
 $\text{parent}[b] = c$

Code Implementation for Disjoint Sets

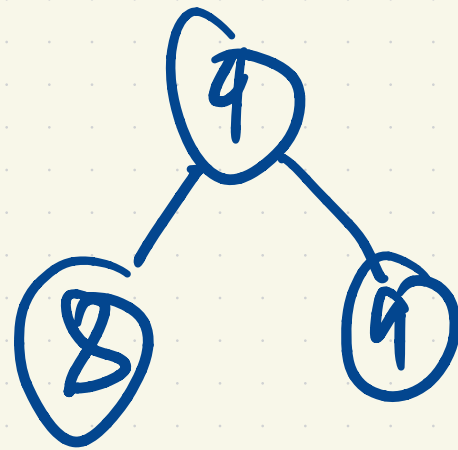
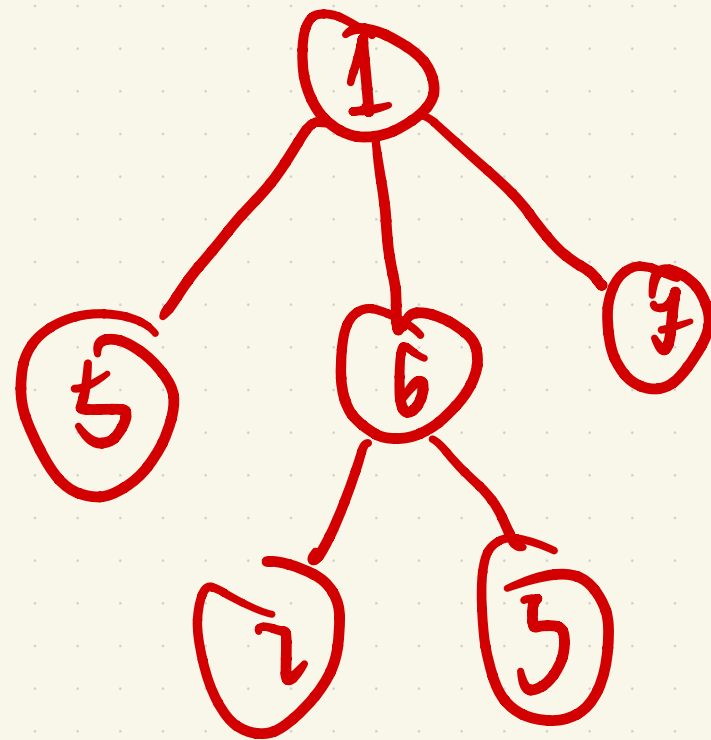
The Union(x,y) function:

```
void Union(int x,int y) {  
    x = FindSet(x);  
    y = FindSet(y);  
  
    if(x != y)  
        parent[x] = y; //node y becomes the representative  
                        // of set where x belongs  
}
```

Handwritten notes:
FindSet(12) = 4
FindSet(6) = 2
parent[1] = 2



- The Union(x,y) function checks first if the elements x and y already have the same root node. If yes, the union is not performed. If no, then the union is performed.
- The Union(x,y) sets the **parent of the representative of x** to be equal to the representative of y. **NOTE: NOT X BUT THE PARENT OF X.**



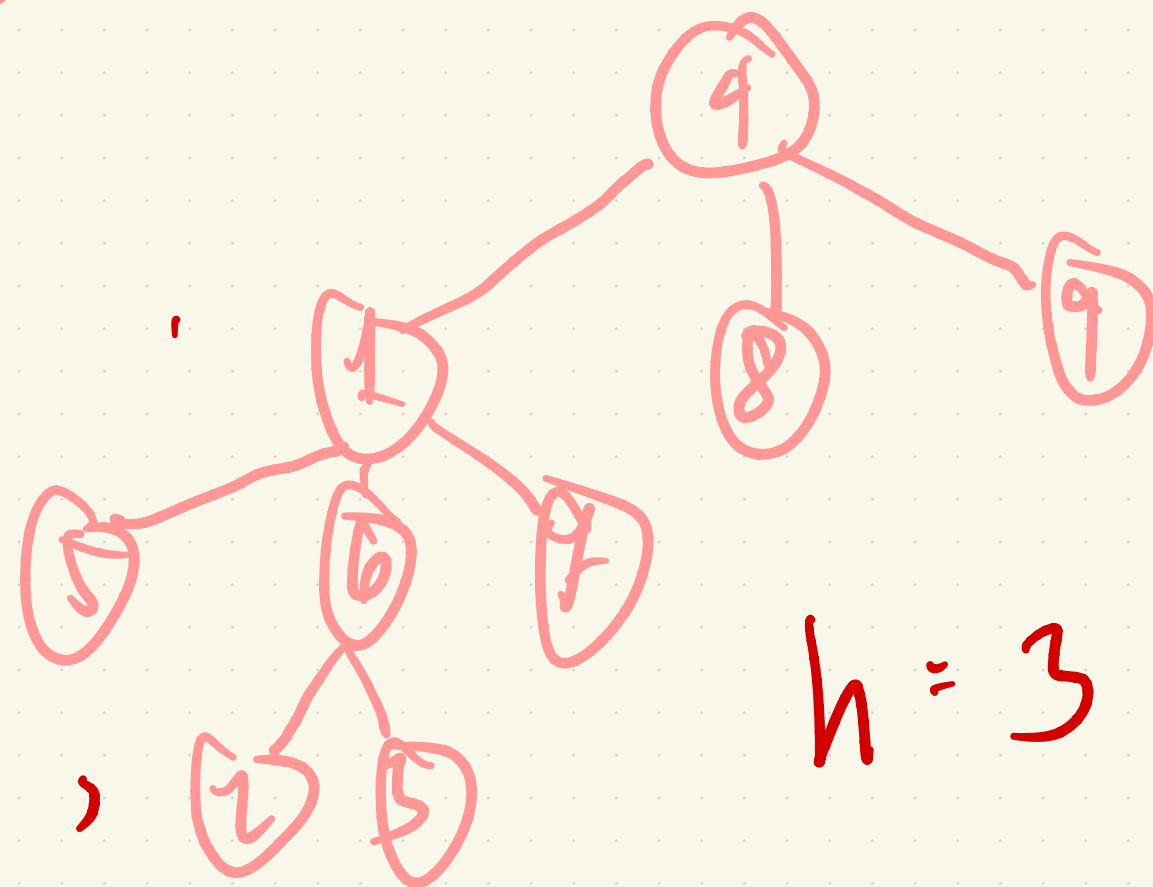
Union (2, 8)

↳ FindSet(1) = 1

↳ FindSet(8) = 4

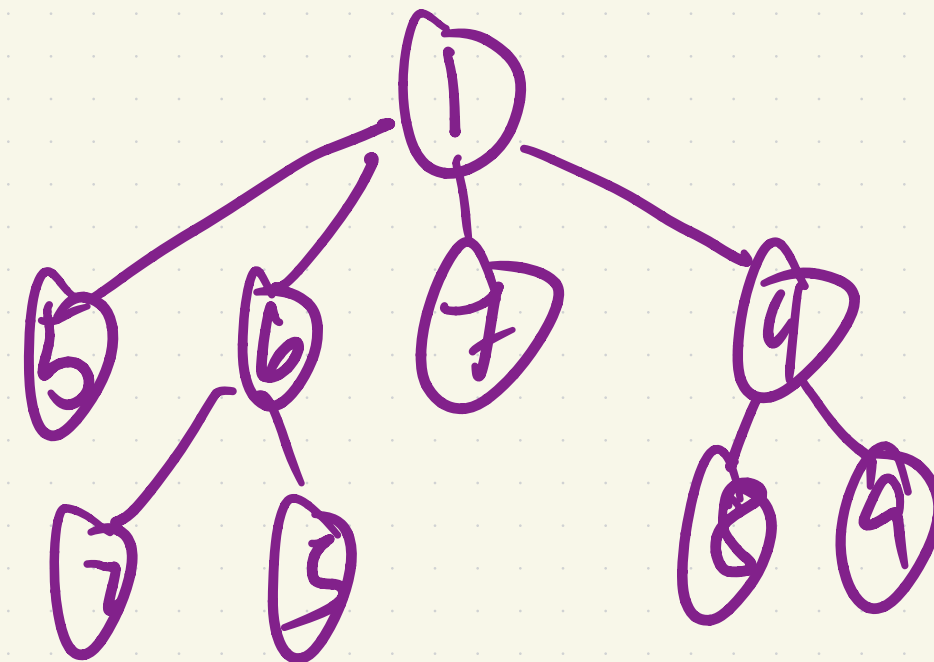
Rank → height / no. of nodes

Approach 1: parent[1] = 4



$h = 3$

Approach 2: parent[4] = 1



$h = 2$

FindSet(x) & Union ⇒ $O(h)$

Code Implementation for Disjoint Sets

But what if the height of x is bigger than the height of y?

- The resulting union would be a taller tree.
- We would need to change a lot of values during FindSet() due to **path compression**.

Solution? We improve Union(x,y) by using a heuristic

- Main Idea: Make the root node of the tree with **smaller height** point to the root of the tree with bigger height. *→ height, no. of nodes, ...*
- Our heuristic? Let's set a value **rank** and we perform **Union by Rank**:
 - Each node has a rank that approximates the logarithm of the size of the subtree at each node.
 - Serves as the upper bound of the height of the node.
 - The root with the smaller rank will be pointed to the root node with higher rank during Union(x,y)

Code Implementation for Disjoint Sets

Union(x,y) by Rank:

```
void Union(int x,int y) {  
    x = FindSet(x);  
    y = FindSet(y);  
  
    if(x != y) {  
        if(rank[x] > rank[y]) {  
            parent[y] = x;  
        } else if (rank[x] < rank[y]) {  
            parent[x] = y;  
        } else {  
            parent[x] = y;  
            rank[y] = rank[y] + 1;  
        }  
    }  
}
```

Note that we also update MakeSet() to include a new array – rank – where all elements are initialized to 0.

Code Implementation for Disjoint Sets

Union(x,y) by Rank:

```
void Union(int x,int y) {  
    x = FindSet(x);  
    y = FindSet(y);  
  
    if(x != y) {  
        if(rank[x] > rank[y]) {  
            parent[y] = x;  
        } else if (rank[x] < rank[y]) {  
            parent[x] = y;  
        } else {  
            parent[x] = y;  
            rank[y] = rank[y] + 1;  
        }  
    }  
}
```

**Rank can be the depth of the
representative of the tree. It can also be the
number of nodes, etc.**

- Disjoint Set DS (Union-Find)
- Implementation
- **Applications**

Sample Application

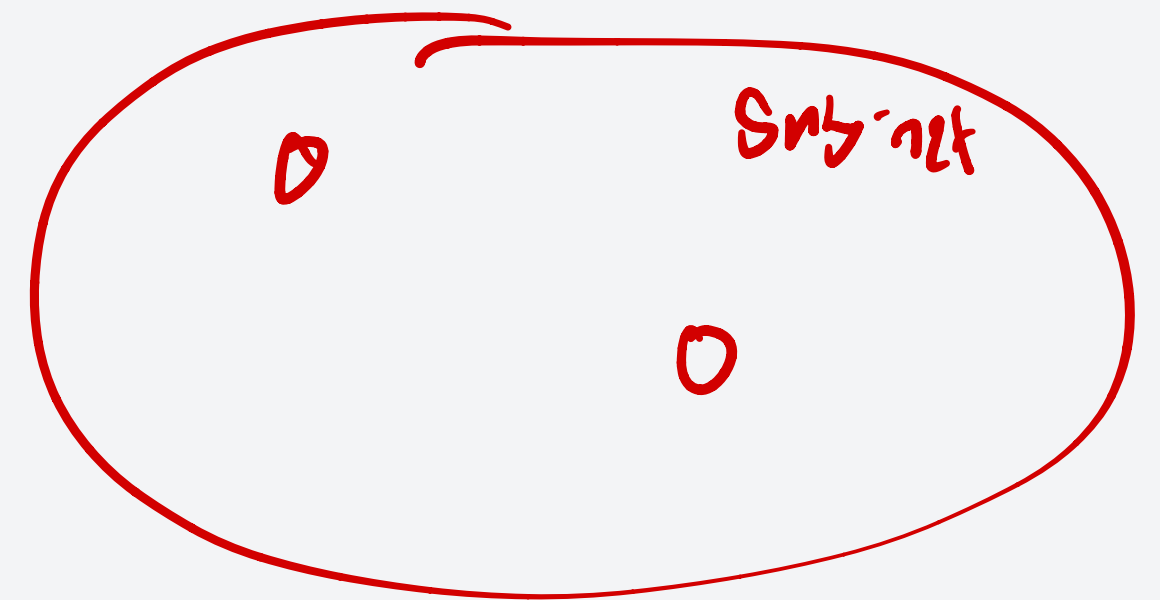
Assumption: $G = (V, E)$ is an undirected graph.

1. Computing the connected components of G

```
ConnectedComponents( $G$ )  
  MakeSet( $V$ )  
  for each edge  $(u, v) \in E$   
    if FindSet( $u$ )  $\neq$  FindSet( $v$ )  
      Union( $u, v$ )
```

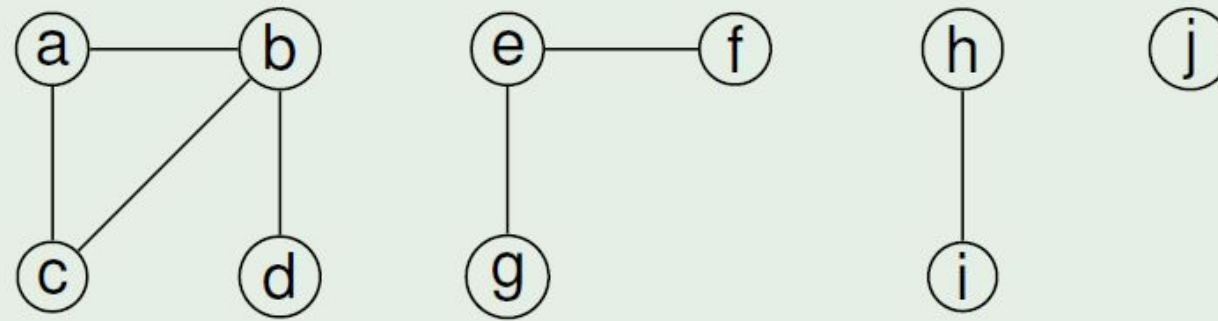
2. Determine if two elements are in the same component

```
SameComponents( $G$ )  
  if FindSet( $u$ ) == FindSet( $v$ )  
    return True  
  else  
    return False
```



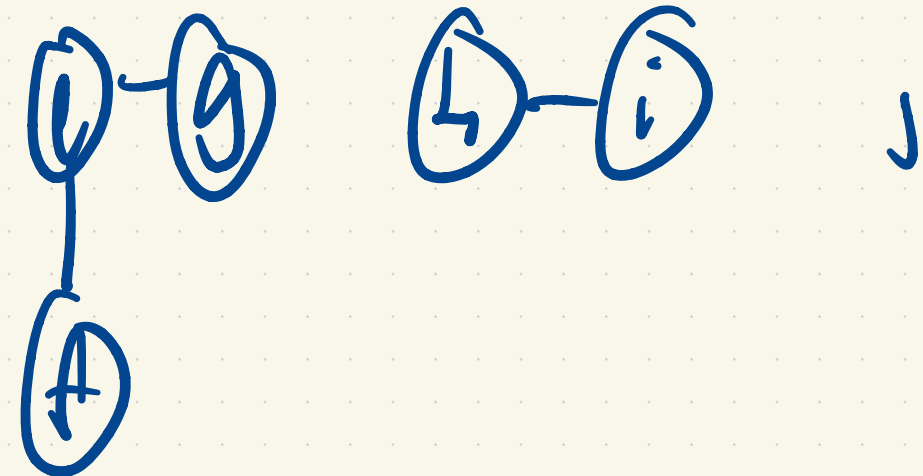
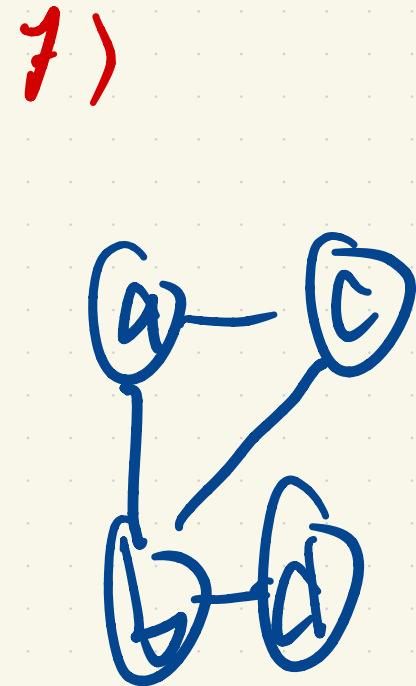
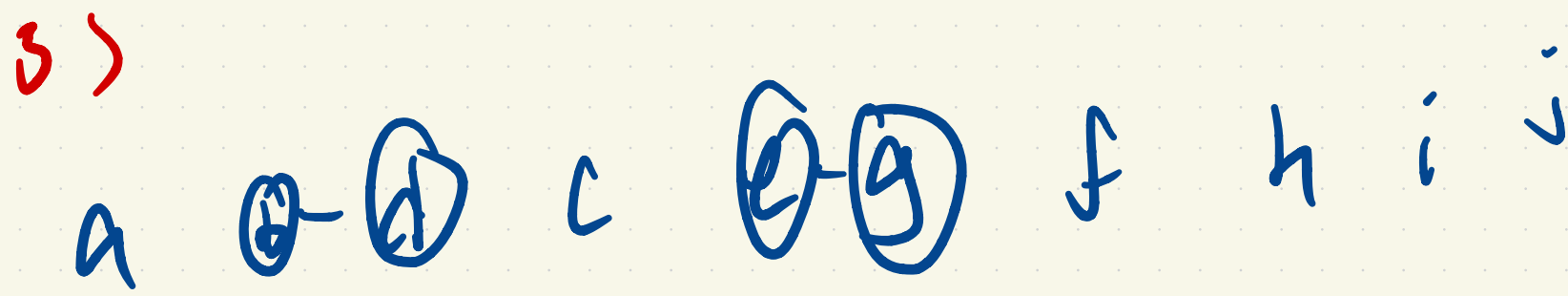
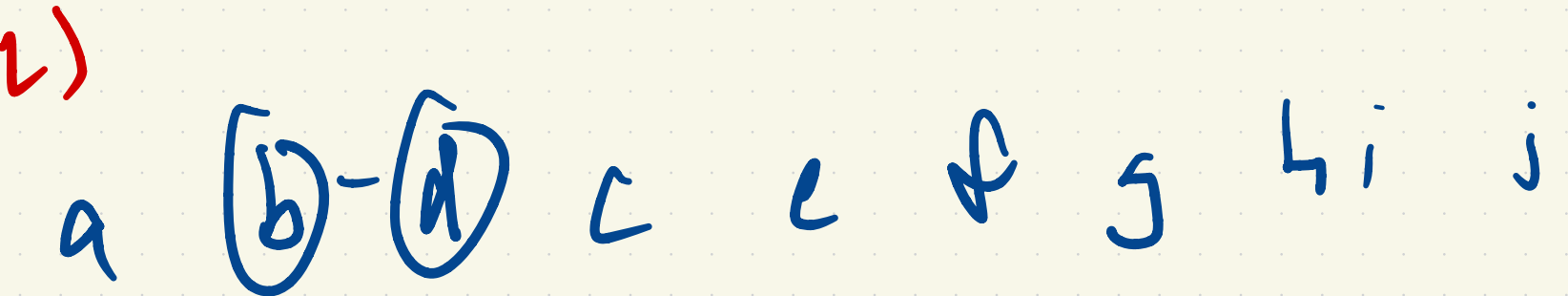
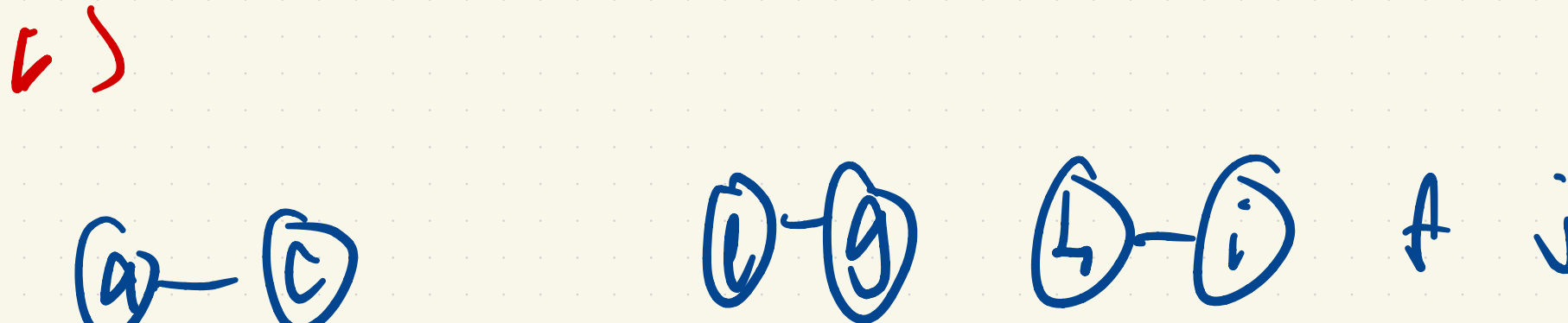
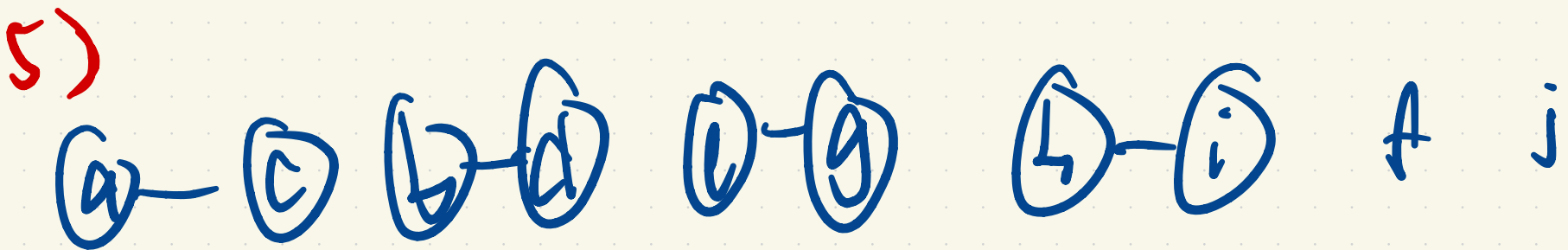
Sample Application

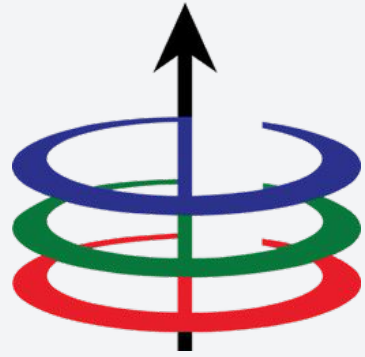
Example (A graph with 4 connected components)



Edge processed	Collection of disjoint sets									
initial sets	{a}	{b}	{c}	{d}	{e}	{f}	{g}	{h}	{i}	{j}
(b, d)	{a}	{b, d}	{c}		{e}	{f}	{g}	{h}	{i}	{j}
(e, g)	{a}	{b, d}	{c}		{e, g}	{f}		{h}	{i}	{j}
(a, c)	{a, c}	{b, d}			{e, g}	{f}		{h}	{i}	{j}
(h, i)	{a, c}	{b, d}			{e, g}	{f}		{h, i}		{j}
(a, b)	{a, b, c, d}				{e, g}	{f}		{h, i}		{j}
(e, f)	{a, b, c, d}				{e, f, g}			{h, i}		{j}
(b, c)	{a, b, c, d}				{e, f, g}			{h, i}		{j}

Edge processed	Collection of disjoint sets									
initial sets	{a}	{b}	{c}	{d}	{e}	{f}	{g}	{h}	{i}	{j}
(b, d)	{a}	{b, d}	{c}		{e}	{f}	{g}	{h}	{i}	{j}
(e, g)	{a}	{b, d}	{c}		{e, g}	{f}		{h}	{i}	{j}
(a, c)	{a, c}	{b, d}			{e, g}	{f}		{h}	{i}	{j}
(h, i)	{a, c}	{b, d}			{e, g}	{f}		{h, i}		{j}
(a, b)	{a, b, c, d}				{e, g}	{f}		{h, i}		{j}
(e, f)	{a, b, c, d}				{e, f, g}			{h, i}		{j}
(b, c)	{a, b, c, d}				{e, f, g}			{h, i}		{j}





Lecture 8

Disjoint Set Data Structure (Union-Find)

EEE 121 2s2526

Steven S. Sison

$(x_1, x_2, x_3, x_4) \rightarrow X_{\text{test}}$

